

La definition de portion reprise

Force Tracker — 13/09/2026, ft - v1203. Quinzieme document de la serie, sous-etape **3-v**. Michel valide la precedente et retrograde explicitement le plan : « *considere le plan comme un ordre de travail, pas comme une source de chiffres ; recompte le perimetre reel le jour meme* » · « *si un proprietaire existe deja, branche l'appelant dessus au lieu de recreer une abstraction* ».

LES DEUX FAITS DE CETTE SOUS-ETAPE

- (1) **La moitie du travail etait deja faite, et le plan ne le savait pas.** Ce qu'il appelle « 3-v » est en realite deux choses : la **definition** de portion (2 copies strictement identiques — extraite ici) et le **nombre** de portions, qui a **deja son proprietaire** et ses deux appelants depuis trois semaines. *La consigne de Michel etait honoree avant meme que la sous-etape commence.*
- (2) **La frontiere qui reste n'est pas seulement invisible : elle est HORS D'ATTEINTE.** La consigne disait « *conserve les temoins de source lorsque la derive serait invisible a l'execution* ». Mesure ici : **aucune fixture ne peut faire rougir un temoin de comportement** sur cette frontiere. Ce n'est plus « difficile a voir », c'est **impossible** — et le seul instrument qui reste est un temoin qui lit le code.

1. Le test d'entree — et pour la premiere fois, le plan dit vrai

Apres **quatre** sous-etapes dont le perimetre ecrit ne resistait pas a la mesure, celui-ci tient sur la moitie qu'il decrit : les deux portes de reprise portaient la meme ligne, **caractere pour caractere** au nom de variable pres. **Exactement 2 copies.**

```
// AVANT – deux portes, la meme ligne, caractere pour caractere

// quickFillFood (« Mes aliments ») :
if(it.u==='portion'){ _afPortionLabel=String(it.portionLabel||'').slice(0,24);
                        _afPortionPoids+=it.portionWeightG>0?it.portionWeightG:0; }

// _afSuggPrendreLocale (recherche du journal) :
if(e.u==='portion'){ _afPortionLabel=String(e.portionLabel||'').slice(0,24);
                    _afPortionPoids+=e.portionWeightG>0?e.portionWeightG:0; }

// APRES – un proprietaire, deux appels
function _afReprendreDefPortion(src){
  const s = src || {};
  if(s.u !== 'portion') return false;
  _afPortionLabel = String(s.portionLabel || '').slice(0, 24);
  _afPortionPoids = +s.portionWeightG > 0 ? +s.portionWeightG : 0;
  return true;
}
_afReprendreDefPortion(it); // quickFillFood
_afReprendreDefPortion(e); // _afSuggPrendreLocale
```

Occurrences dans le code servi, hors commentaires : `_afReprendreDefPortion` **3** (1 declaration + 2 appels) · definition ecrite en dur hors du proprietaire : **0**.

2. L'autre moitie etait deja faite

ce que le plan appelle « 3-v »	etat reel, mesure le jour meme
la definition : quelle portion, combien elle pese	2 copies strictement identiques — c'est ce qui est extrait ici
le nombre de portions	X deja fait : <code>_afReprendrePortions(n)</code> existe, et les deux portes y sont branchees depuis trois semaines. 3 occurrences (1 declaration + 2 appels)

CE QUE CA DIT DU RAPPORT ENTRE UN PLAN ET UN DEPOT

Le plan a ete ecrit le 12/09. Entre-temps, **d'autres versions ont travaille sur les memes lignes** — et le proprietaire du nombre de portions a ete pose par l'une d'elles, pour une raison qui n'avait rien a voir avec ce chantier.

=> **Un plan ne sait pas ce qui a ete fait entre-temps.** Il decrit un depot a une date, et le depot bouge. C'est la cinquieme fois de suite qu'un perimetre ecrit diverge du code — mais pour la premiere fois l'ecart est dans le **bon** sens : *il restait moins a faire que prevu, pas plus.*

3. Ce que le propriétaire fait, et ce qu'il refuse

cas	resultat, aux DEUX portes
portion complete (part / 120 g)	repris tel quel
portion sans poids	etiquette reprise, poids a 0 — on ne devine pas un poids (R29)
portion sans nom	poids repris, etiquette vide
en grammes	X rien n'est repris — une definition de portion n'appartient pas a un aliment pese
poids nul ou negatif	0, jamais une valeur de remplacement
portion AVEC un pour-100 g	* repris quand meme — c'est le cas qui porte tout le §4

=> Les deux portes donnent exactement la meme table sur les 6 cas — c'est ce qui rend l'extraction legitime, et un temoin le fige.

4. L'asymetrie de condition — et pourquoi elle est hors d'atteinte

```
// LES DEUX LIGNES, CHEZ LES APPELANTS – adjacentes, et PAS la meme condition
_afReprendreDefPortion(it); // la DEFINITION -- aucun garde
if(!_bcNutr && +it.q>0 && it.u==='portion' && ...) _afReprendrePortions(+it.q);
    ^^^^^^^^^ le NOMBRE -- garde par !_bcNutr

// LA CHAINE MESUREE, LIGNE A LIGNE :
// 1. quickFillFood entre par _afOublierAliment() -> _bcNutr = null
// 2. le seul site qui POSE _bcNutr est garde par -> it.u !== 'portion'
// 3. la branche else -> _bcNutr = null
//
// => si it.u === 'portion', _bcNutr vaut TOUJOURS null a la ligne 2.
// Le !_bcNutr ne peut JAMAIS bloquer. Verifie a la sonde sur 6 cas :
// bcNutr: false partout -- Y COMPRIS sur la fixture portant un per100,
// celle qui etait precisement concue pour le poser.
```

LA CONSIGNE DE MICHEL, POUSSEE D'UN CRAN PAR LA MESURE

Il a ecrit : « une derive de conception peut etre invisible a l'execution ». Ici c'est plus fort. La ligne du **nombre** porte un garde qui **ne peut pas etre faux** : le bloc qui poserait _bcNutr se refuse lui-meme les aliments en portions.

=> **Aucune fixture ne peut atteindre l'etat qu'un temoin de comportement devrait mesurer.** Les deux temoins de source de ce bloc ne sont donc pas un supplement de prudence : ils sont **les seuls possibles**. La mutation qui absorbe le garde dans le propriétaire rend **1 rouge, uniquement lui** — tout le reste passe au vert.

X **Et ce garde redondant n'est PAS retire.** Il est inutile aujourd'hui ; il serait **le seul** le jour ou quelqu'un retirerait le u !== 'portion' d'a cote. *Un garde inutile aujourd'hui peut etre le seul garde demain.* Mesure, ecrit dans le journal de test, **non corrige** — une extraction ne change aucun comportement.

5. La sonde : elle POSAIT, elle ne relisait pas

```
// LA SONDE POSAIT, ELLE NE RELISAIT PAS – un 3e visage du meme piege

// Depuis des versions, la sonde ECRIT ces deux variables en fixture :
_afPortionPoids = 125; _afPortionLabel = 'steak'; // lignes 95 / 116 / 123

// ... mais AUCUNE des 19 cles ne les relisait apres avoir conduit une porte.
// => l'instantane serait reste identique quoi qu'on fasse a la ligne visee,
// donc il aurait valide n'importe quelle extraction.

// ft-v1199 : conduire n'est pas observer.
// ft-v1203 : POSER encore moins. -> 19 cles, puis 21, AVANT le BEFORE.
```

ET MA PREMIERE FIXTURE DE PERIMETRE SUPPOSAIT CE QU'ELLE VOULAIT MESURER

J'avais recopie le cas « avec pour-100 g » de la sous-etape precedente en croyant qu'il poserait _bcNutr. **Vrai pour un aliment en grammes, faux pour un aliment en portions.**

=> La sonde **LIT** désormais _bcNutr au lieu de le supposer — et c'est exactement ce qui a fait tomber la decouverte du §4. *Une fixture qui suppose l'etat qu'elle veut mesurer ne le mesure pas : elle le decrete.*

6. Ce qui reste volontairement dehors

ce qui ne bouge pas	la raison, mesurable
la ligne du NOMBRE chez les appelants	2 copies , gardees par <code>!_bcNutr</code> . L'absorber changerait la condition d'appel — un temoin l'interdit
<code>_afReprendrePortions</code>	intacte, 3 occurrences . C'est le proprietaire du nombre : on ne lui en oppose pas un second
les jumelles <code>_efPortionLabel / _efPortionPoids</code>	meme grandeur, AUTRE ecran (l'edition). Separation documentee dans le code depuis le 10/09. Les fondre serait une refonte de deux ecrans , pas une extraction
le garde d'unite <code>u==='portion'</code>	il est dans le proprietaire, parce qu'il fait partie de son metier : <i>reprendre une definition de portion</i> . Le retirer ferait passer celle d'un aliment pese

//\ **5e ecart de perimetre du plan, decouvert au passage — il porte sur la sous-etape SUIVANTE**. Le plan annonce 4 sites pour 1b-v. Mesure : `_afSetUnite` n'ecrit `NI _afPortionLabel NI _afPortionPoids` (**0 fois**) ; les deux portes de reprise sont faites par 3-v ; et `openEditFood` ecrit les jumelles `_ef*`. **Il ne reste qu'UN site, qui ecrit d'autres variables** — le test d'entree s'appliquera.

7. Les mesures

preuve	resultat
Instantane, avant / apres	identique octet pour octet , sha256 8d352131a8cde6cf — sonde etendue de 19 a 21 cles AVANT le BEFORE
Temoins	19 — bloc CCC, dont 7 de source
Controle negatif	11 mutations, toutes mordent sur leur PROPRE temoin ; controle sain a 0 rouge avant ET apres
Passe complete	3699 / 3699 — <i>lu dans le journal de la passe, pas ecrit a la main</i> · total predit = obtenu (3680 + 19)
Autres bancs	calculs 339/339 · muscles 241/241 · croises 50/50 · dates 9/9 · donnees : aucun trou nouveau
Ecran	rien ne change

//\ **Et une mutation avait d'abord un ancrage INVALIDE** : `const s = src || {};` existe **6 fois** dans le fichier — elle frappait un autre proprietaire. *Plus on extrait de proprietaires, plus les motifs se ressemblent d'une fonction a l'autre*, et une mutation mal placee est indiscernable d'un temoin aveugle. Reancree sur deux lignes contigues propres a celui-ci.

8. Etat du decoupage

Sous-etapes restantes avant le hub	1 — sur 10 au plan, 8 livrees, 1 ecartee : 1b-v seule
Dependance de 1b-v	OUI levee — elle attendait 3-v, qui est livree. Mais son perimetre ecrit est faux (voir §6) : a recompter le jour meme
Defaut reel decouvert ?	* oui — le garde qui ne peut jamais bloquer. Mesure, cause documentee, NON corrige , en attente d'un feu vert separe
Le hub et la douane	apres 1b et 3, consigne inchangee
Favoris entre onglets, ecart 48,3 / 48	ouverts, non corriges
Historique, migrations, harmonisations produit	non touches

Rollback : un `git revert` du commit.

Document genere depuis le code servi — tous les decompotes cites sont recomptes a chaque generation (3 occurrences du proprietaire, 0 ecriture en dur restante, 3 du proprietaire du nombre, 2 lignes du nombre chez les appelants, 0 ecriture dans `_afSetUnite`, 21 cles de sonde, 19 temoins, 1 sous-etape restante). **Dix-neuf gardes refusent de produire si un fait tombe** — dont un qui verifie que `_bcNutr` n'est **pas** entre dans le proprietaire (la derive hors d'atteinte), un que le proprietaire ne touche **pas** `_afPortions` (le nombre a deja le sien), un que les jumelles `_ef*` ne sont pas absorbees, et un que la sonde **LIT** `_bcNutr` au lieu de le supposer. Le total de la passe est **lu dans son journal**, jamais ecrit a la main. Source : `tools/gen_1203_pdf.py`.